

Architecting Reproducible Science: A Practical Path Beyond the Notebook

Software engineering practices for reliable HPC workflows

Fernando Garzon, SDSC Summer Institute 2026

[linkedin.com/in/fgarzonm](https://www.linkedin.com/in/fgarzonm) - github.com/ofgarzon2662

SDSC
SAN DIEGO SUPERCOMPUTER CENTER

UC San Diego

The model worked. The workflow failed.

The model had strong back-tests and worked on her laptop.

The demonstration depended on cells executed in a particular order.

A skipped optimization cell caused the notebook to exhaust memory.

The science did not suddenly become wrong; the execution was fragile.

Takeaway: Hidden state can turn a successful experiment into a failed demonstration.



Notebook state hides dependencies

Execution order: Run All produces a different result.

Variables in memory: a clean kernel raises an error.

Local paths and data: the code works only on one laptop.

Library versions: reinstalling changes the result.

Work during import: tests and tools become slow or fail.

Takeaway: A workflow that depends on memory or history is not yet repeatable.





Four different workflow guarantees

Terminology varies by field. In this talk:

Repeatable: the same inputs and environment produce the result again.

Reproducible: another person can rebuild and repeat the workflow.

Portable: the same entry point runs on another platform.

Scalable: the algorithm and resource plan use larger systems effectively.

Takeaway: Packaging improves repeatability and portability; it does not guarantee scale.



Notebooks and packages do different jobs

Use a notebook to:

Explore data and hypotheses.

Keep equations, code, plots, and reasoning together.

Use a package to:

Expose stable functions and declare dependencies.

Support tests, automation, and multiple entry points.

Takeaway: The durable workflow is notebook plus package.

Move reusable code out of notebook state

Turn key calculations into functions with explicit arguments.

Pass paths and configuration instead of reading local globals.

Keep downloads, training, and plotting behind entry points.

Return values instead of relying on variables left in memory.

Guard expensive scripts with `if __name__ == "__main__":`.

Takeaway: The notebook tells the story; the package provides the implementation.



The skydiver model has a testable contract

`terminal_velocity(mass, drag, gravity=9.81)`
`velocity_at_time(time, mass, drag)`

Mass, drag, and gravity must be positive.

Velocity starts at zero and approaches a finite limit.

More mass raises the limit; more drag lowers it.

Takeaway: Scientific assumptions become behavior that software can check.





nbdev exports selected notebook cells

```
#| default_exp physics  
#| export  
def terminal_velocity(...):
```

```
...
```

nbdev-export

The notebook remains the source for exported functions.

Generated modules are committed and reviewed.

Takeaway: A command replaces the memory of which cells matter.

Every package artifact has a home

nbs/00_skydiver.ipynb # explanation + source
skydiver/physics.py # exported functions
skydiver/cli.py # command-line entry
tests/test_physics.py # scientific checks
pyproject.toml # dependencies + build
README.md # install + run

Takeaway: A collaborator can find the model, tests, and instructions without asking.



Test first in a fresh process

```
python -m venv .venv
python -m pip install -e ".[dev]"
nbdev-export
pytest -q
skydiver --mass 80 --drag 0.26
```

54.94 m/s terminal velocity; 14.82 s to reach 99%.
Takeaway: The CLI no longer depends on the notebook kernel.



Tests should protect scientific meaning

Known value: example inputs produce the expected terminal velocity.

Invariant: the limit is finite and has the expected sign convention.

Relationship: more mass raises the limit; more drag lowers it.

Boundary: invalid physical parameters raise a clear error.

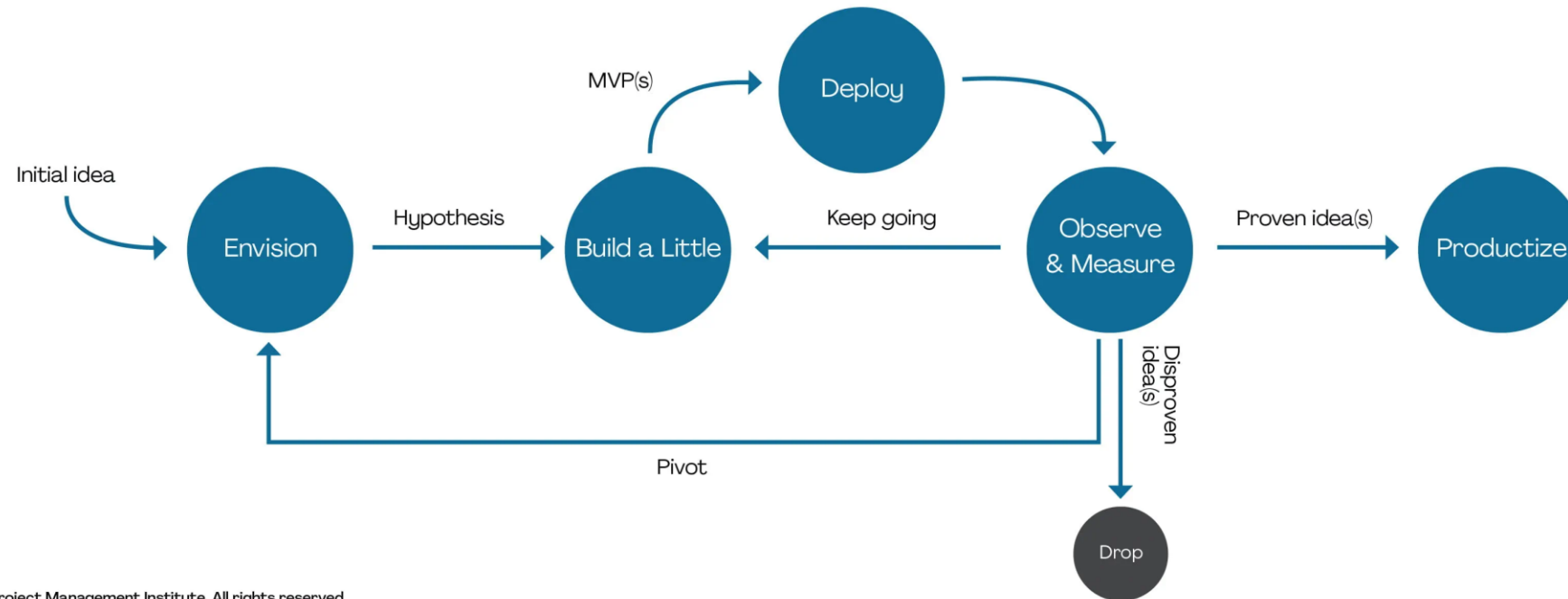
Interface: scalar and array inputs behave predictably.

Takeaway: Coverage says code ran; scientific tests say claims still hold.



CI repeats the clean-room workflow

checkout -> install -> export -> test -> build wheel
A failed gate stops the release before an expensive or public step.



©Project Management Institute. All rights reserved.

Image reference: <https://www.pmi.org/disciplined-agile/lifecycle/exploratory-lifecycle>

A wheel is a versioned Python artifact

python -m build --wheel

A wheel contains package code and installation metadata.

A version identifies the implementation that produced a result.

CI builds the artifact only after tests pass.

Deployment installs the wheel instead of copying loose files.

Takeaway: Build once, identify the version, and promote the same artifact.





Containers record the runtime

FROM python:3.12-slim

```
RUN python -m pip install /opt/skydiver  
ENTRYPOINT ["python", "-m", "skydiver.cli"]
```

The package records Python code and dependencies.

The image records the OS-level runtime and entry point.

Build locally or in CI, not on an HPC login node.

Takeaway: A container preserves the environment; it does not validate the science.

One entry point can travel everywhere

notebook -> package -> tests -> wheel -> container

laptop / CI / HPC

The calculation is not rewritten for each destination.

Configuration changes; the application interface does not.

Logs and outputs can be compared across environments.

Takeaway: Portability comes from a stable entry point and recorded runtime.



The scheduler owns HPC resources

```
#SBATCH --time=00:10:00 --mem=8G --gpus=1
```

```
singularity exec --nv model.sif python -m your_package.train
```

Request account, partition/QOS, time, memory, CPUs, and GPUs.

Use --nv only for an allocated NVIDIA workload.

Check current SDSC guides and module spider before submitting.

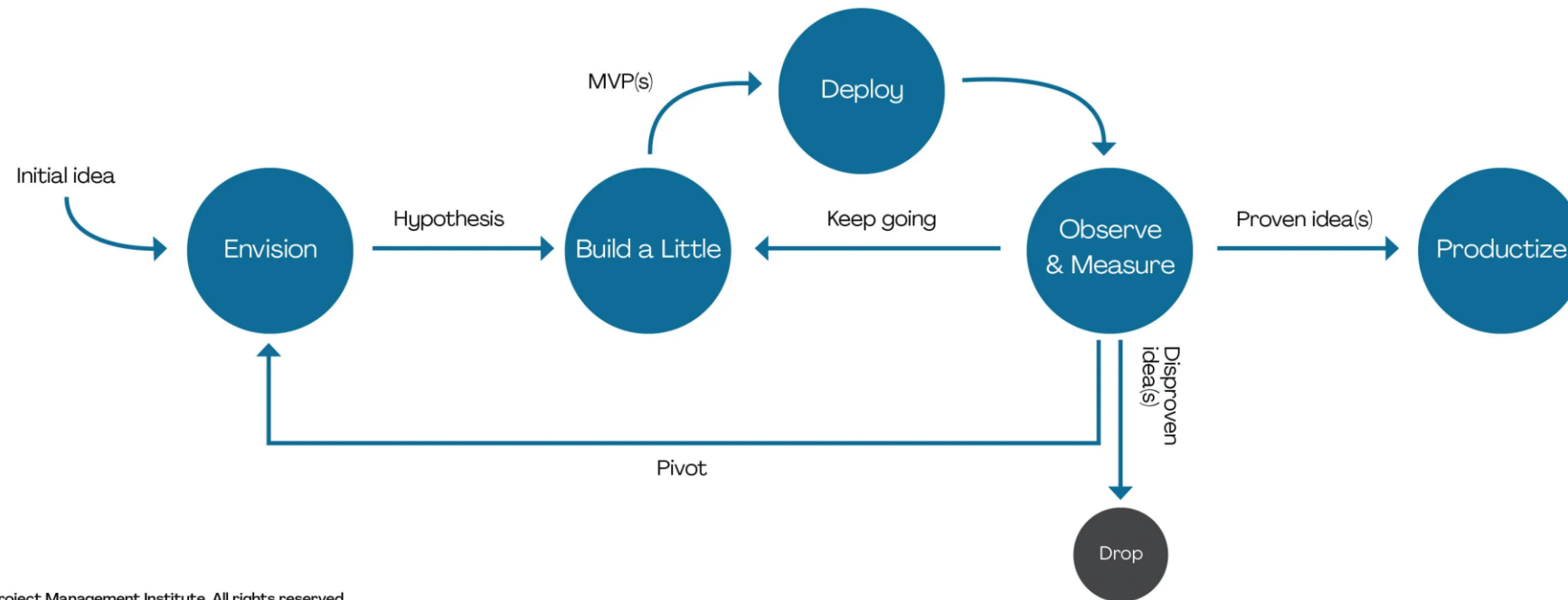
Takeaway: The tutorial is CPU-only; GPU files are templates for the ML workflow.



MLOps is a feedback loop

package -> test -> build -> deploy -> observe -> improve

The goal is less uncertainty between experiment and every later execution.



©Project Management Institute. All rights reserved.

Image reference: <https://www.pmi.org/disciplined-agile/lifecycle/exploratory-lifecycle>

The model stayed. The workflow changed.

Make execution order explicit.

Turn scientific assumptions into tests.

Record dependencies, versions, inputs, and entry points.

Build one artifact and run it consistently.

Request HPC resources explicitly and measure before scaling.

Keep notebooks for exploration and explanation.

Takeaway: Reproducibility is a workflow, not a copy of the notebook.





Questions, then the tutorial

After Q&A:

Open tutorial/README.md.

Export the skydiver package.

Run six tests.

Execute the CLI from a fresh process.

Closing prompt:

What scientific assumption in your code deserves a test before you spend hours on an HPC system?